Attendance



When Linux Lies

Detecting Kernel Rootkits Beyond Conventional Tools

‹talk› About Me

- Felix/ Fai Yew
- Stats (Professional Stuff)
  - Recent Cloudflare Technical Support Engineer Intern
  - NUS Information Security
  - CCNA, AWS CP, GPEN, OSCP, GCFA (Ongoing!)
  - Into Infra/Blue Teaming domains
- Stats (Non-professional Stuff)
  - music lover (jpop larper), bass + guitar + drums player
  - weeb, ig influencer wannabe



About Today's Talk

What should you already know?

- Familiarity with Linux
- Basic knowledge on Operating Systems
- Basic Understanding of C & Python
- Interest in Cybersecurity & OS Security



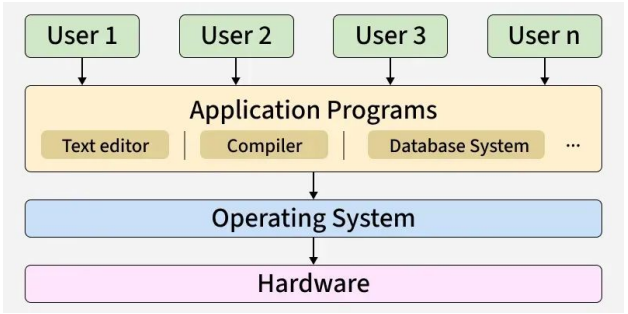
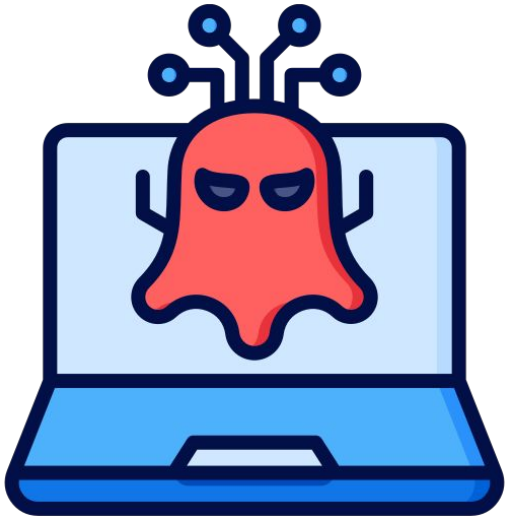


Table of Contents

- Part I: Introduction to Linux Kernel Rootkits
- Part II: Linux Kernel & Syscall Operations
- Part III: Common Linux Kernel Rootkit Mechanisms
- Part IV: Rootkit Defence Tools & POC
- Part V: Where Trust Ends

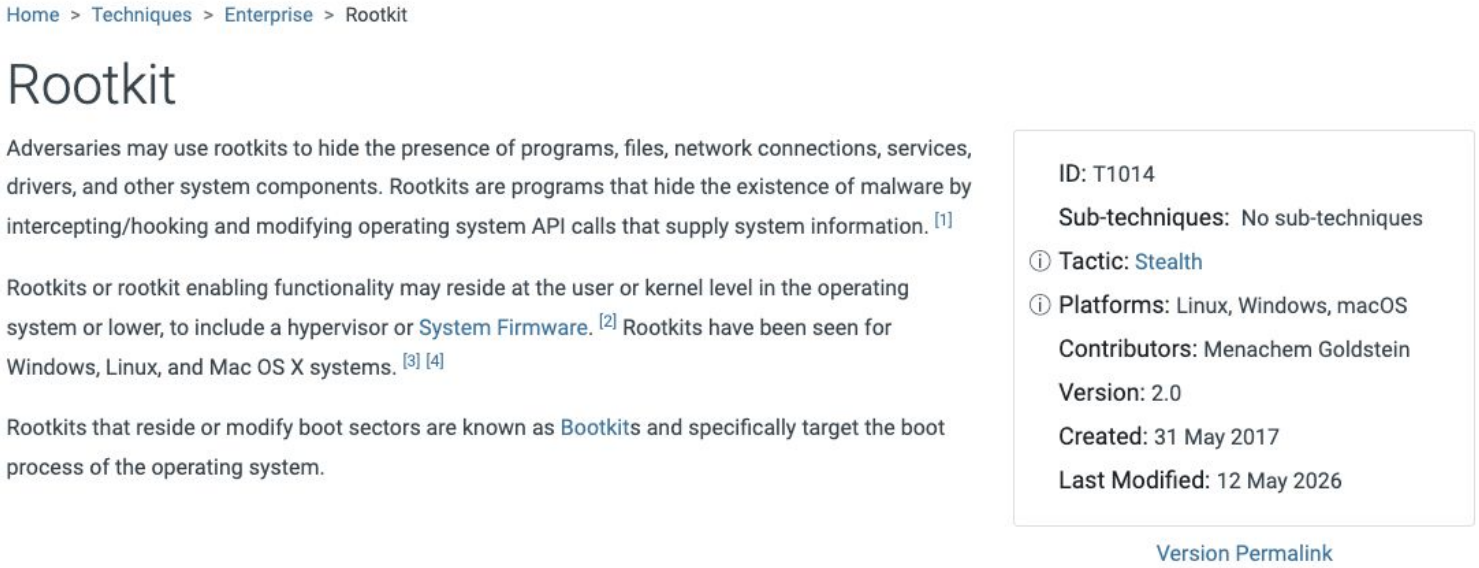


Part I: Introduction to Linux Kernel Rootkits



What are Rootkits?

- “A collection of tools that a hacker uses to mask intrusion and obtain Administrator-level access to a computer or computer network” - SANS



Home > Techniques > Enterprise > Rootkit

Rootkit

Adversaries may use rootkits to hide the presence of programs, files, network connections, services, drivers, and other system components. Rootkits are programs that hide the existence of malware by intercepting/hooking and modifying operating system API calls that supply system information. [1]

Rootkits or rootkit enabling functionality may reside at the user or kernel level in the operating system or lower, to include a hypervisor or System Firmware. [2] Rootkits have been seen for Windows, Linux, and Mac OS X systems. [3] [4]

Rootkits that reside or modify boot sectors are known as Bootkits and specifically target the boot process of the operating system.

ID: T1014

Sub-techniques: No sub-techniques

ⓘ Tactic: Stealth

ⓘ Platforms: Linux, Windows, macOS

Contributors: Menachem Goldstein

Version: 2.0

Created: 31 May 2017

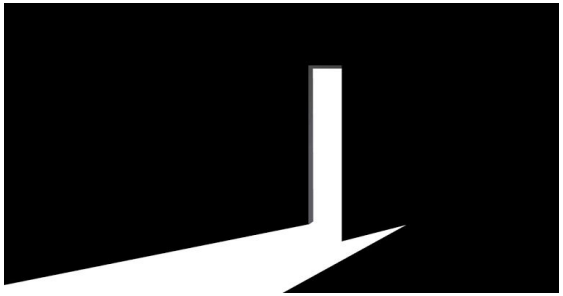
Last Modified: 12 May 2026

Version Permalink

What are Rootkits?

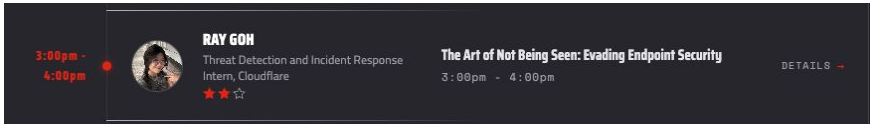
Functions can include:

- Hiding Files, Processes, Network Connections, etc.
- Provide Unauthorised Access (Backdoors)
- Eavesdropping tools (Keyloggers)



Who builds them?

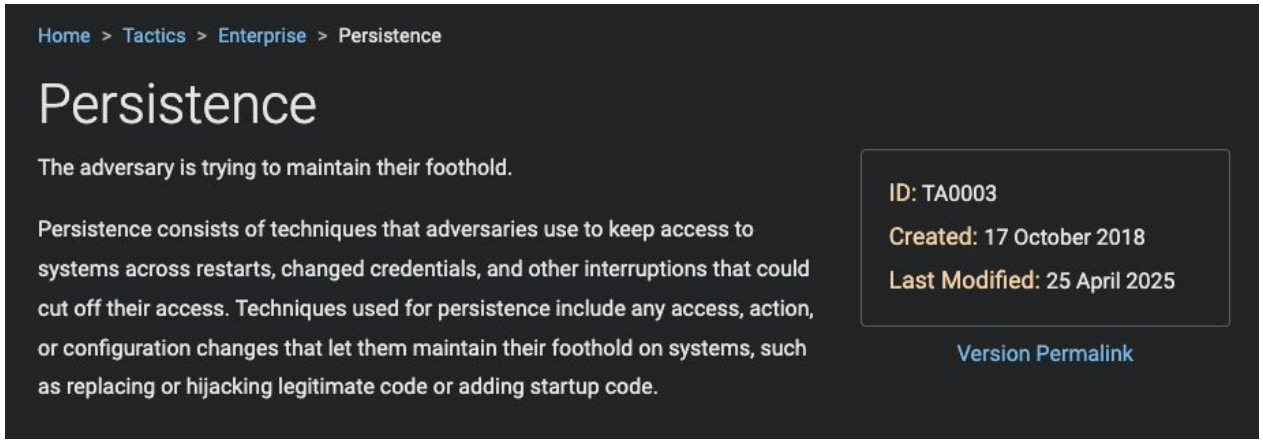
- Software “Features” / Anti-Cheating →
- State-Linked Sabotage/ Espionage (e.g. Stuxnet)
- Cybercrime



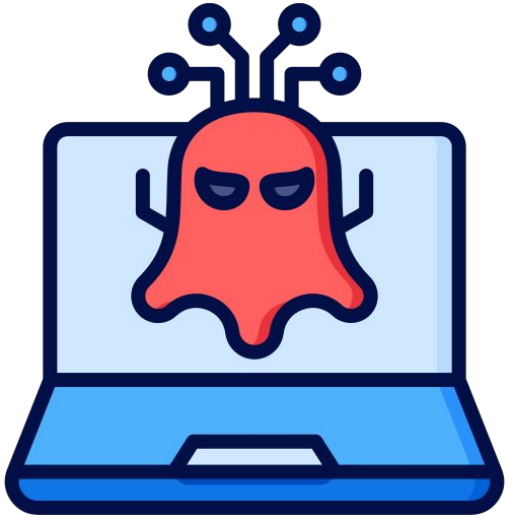
End Goal: Maintain Persistence & Privileges

Attackers use rootkits for persistence mechanisms

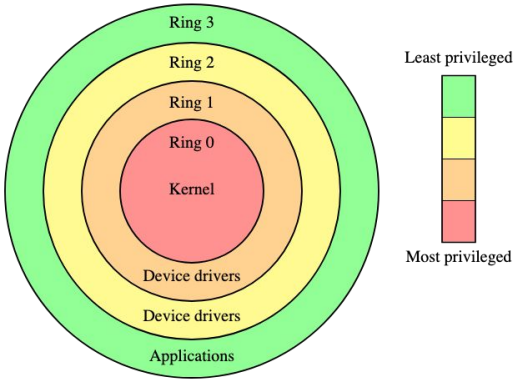
- Maintain access after gaining root privileges.
- May persist across reboots and updates (Requires persistence mechanism)



Linux LKM Rootkit Quick Demo



Part II: Linux Kernel & Syscall Operations



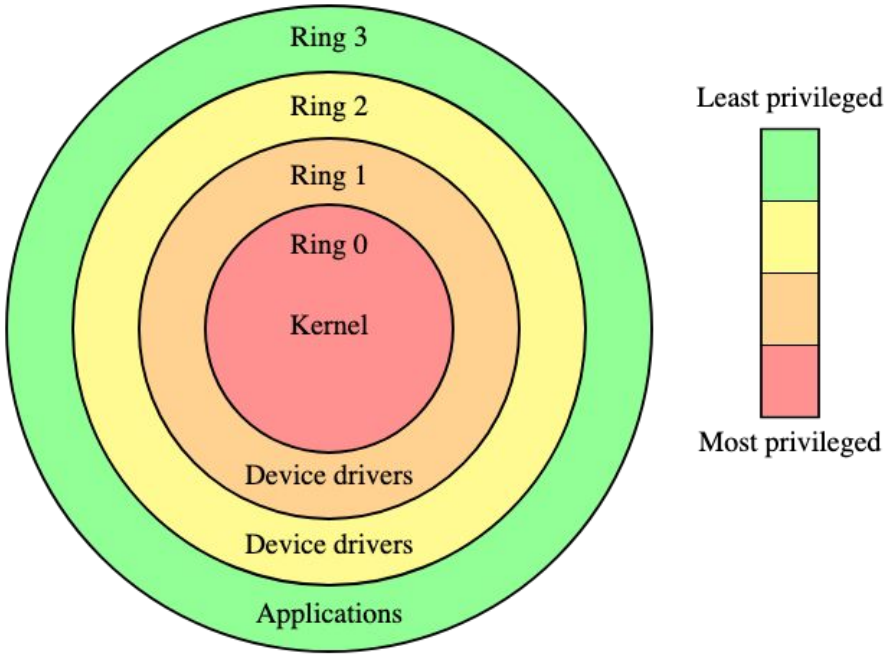
User Space & Kernel Space

Ring 3: User Space

- User-level programs (Applications, Utilities such as `ls`, `cat`)
- Can directly access memory permitted by their virtual address space.
- Cannot normally access kernel memory or execute privileged instructions.
- Request kernel services through **system calls.**

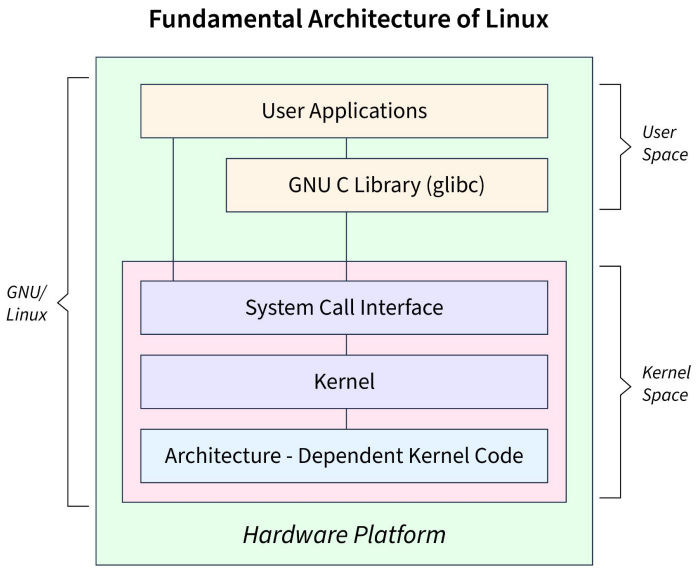
Ring 0: Kernel Space (System Space)

- Access to all system resources
- Kernel Programs & Modules (Drivers)
- Interact with hardware components (RAM, Hard Disks)



Syscall Interface

- **System Calls** let user-space programs request Kernel services
- The kernel dispatches the request to a handler
- Enter the kernel, execute the requested operation, then return control to user space.



SCALER Topics

System Calls (Syscalls)

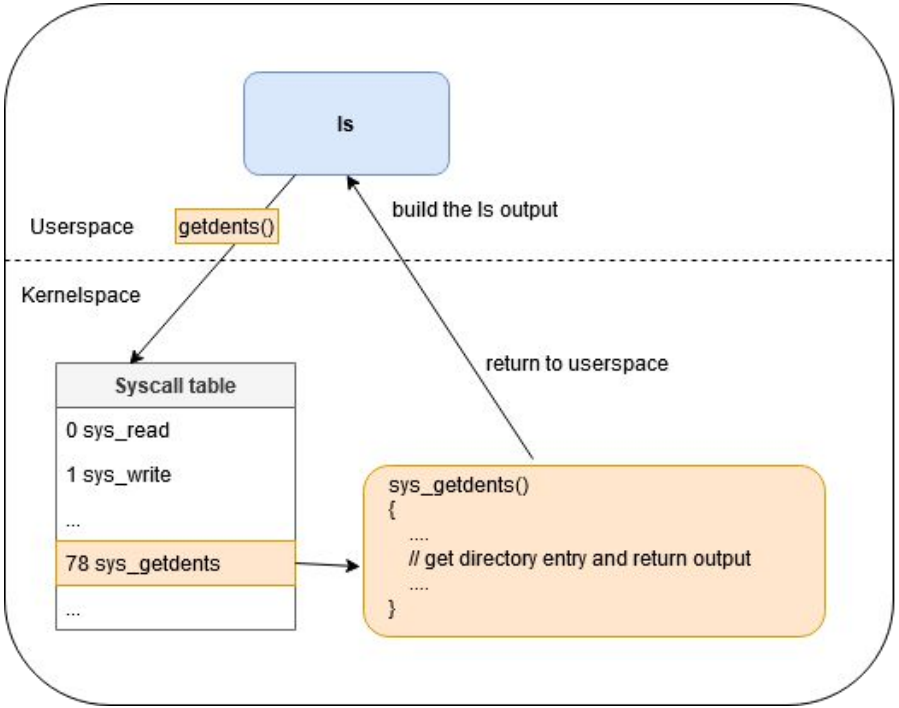
Regular Library Call

- e.g. `strlen()`

System Call

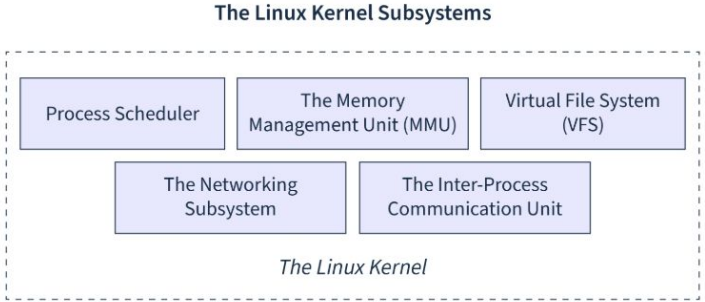
- e.g. `getdents64()`
- User-to-kernel privilege transition with `SYSCALL` instruction
- Jump to kernel, execute, CPU return control to user space (Ring 3)

Traditional table-based dispatch model



Loadable Kernel Modules (LKM)

- Proper way to modify kernel
  - `.ko` object files contain code that extends the kernel.
- Installed modules reside in `/lib/modules/$(uname-r)`
- Query information from `/proc/modules` and `/sys/module/`
- `kmod` utility (Used by package managers)
  - `insmod`: Insert kernel module
  - `rmmod`: Unload kernel module
  - `modprobe`: Module management tool
  - `lsmod`: List all loaded kernel modules



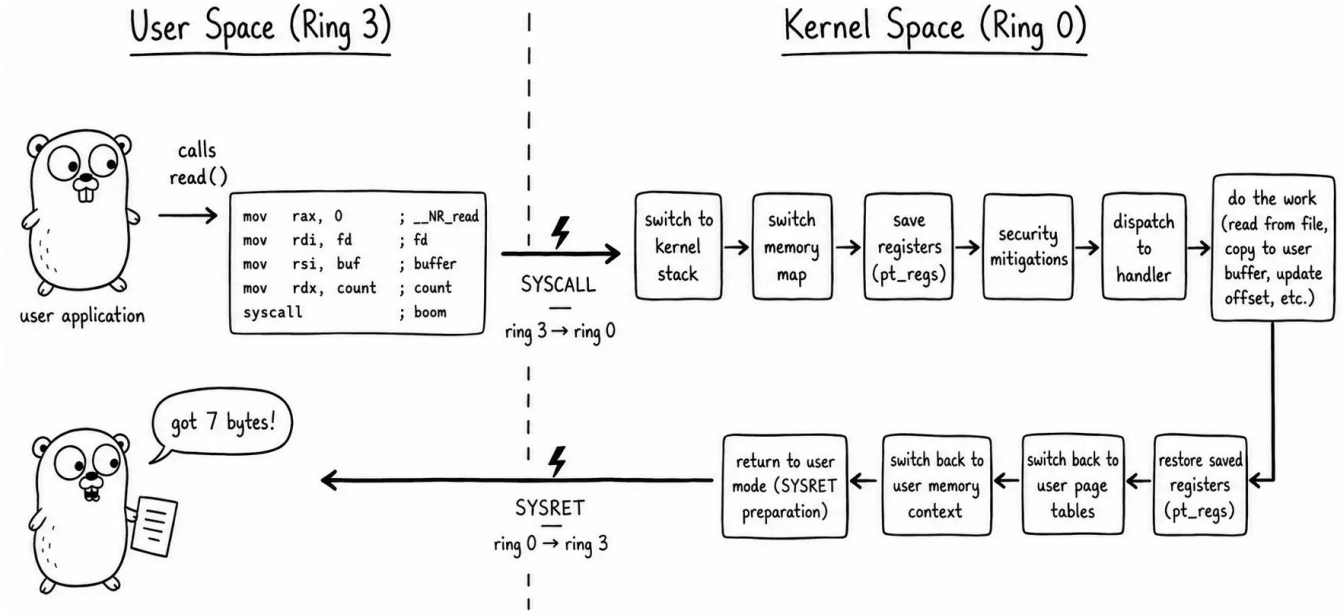
strace

Trace syscalls with `strace`

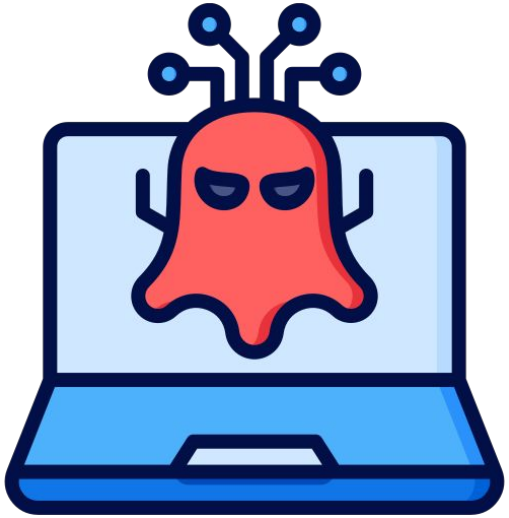
```
felix@ubuntu:~/STANDCON$ strace -e trace=openat,read,write,close cat /etc/passwd
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/etc/passwd", O_RDONLY) = 3
read(3, "root:x:0:0:root:/root:/bin/bash\n"..., 131072) = 2907
write(1, "root:x:0:0:root:/root:/bin/bash\n"..., 2907root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

Syscalls Summary

TLDR: Hardware-mediated crossing from ring 3 into ring 0!



Part III: Common Linux LKM Rootkit Mechanisms



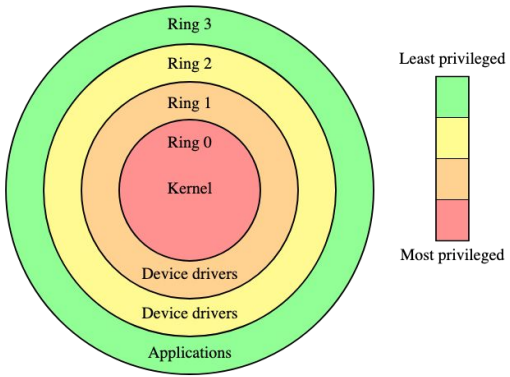
Types of Rootkits

User Space

- Replace selected application binaries, inject code, or intercept library calls.
- Easy to implement
- Detected with Hash Checks, Checking Symlinks, Integrity checks for injections etc.)
- e.g. T0rnkit, Azazel, Jynkkit

Kernel Space

- Change kernel structures & code
- Better concealment
- Can modify binaries, functions, OS syscalls
- e.g. Diamorphine



LKM Rootkit Mechanisms

List of Common LKM Rootkit Mechanisms:

1. Function Pointer Hooking (Syscall Table)
2. Run-time Kernel Code Patching
3. Direct Kernel Object Manipulation (DKOM)
4. Others (`Kprobes`, `Ftrace`, etc.)



LKM Loading: Privilege and Compatibility Requirements

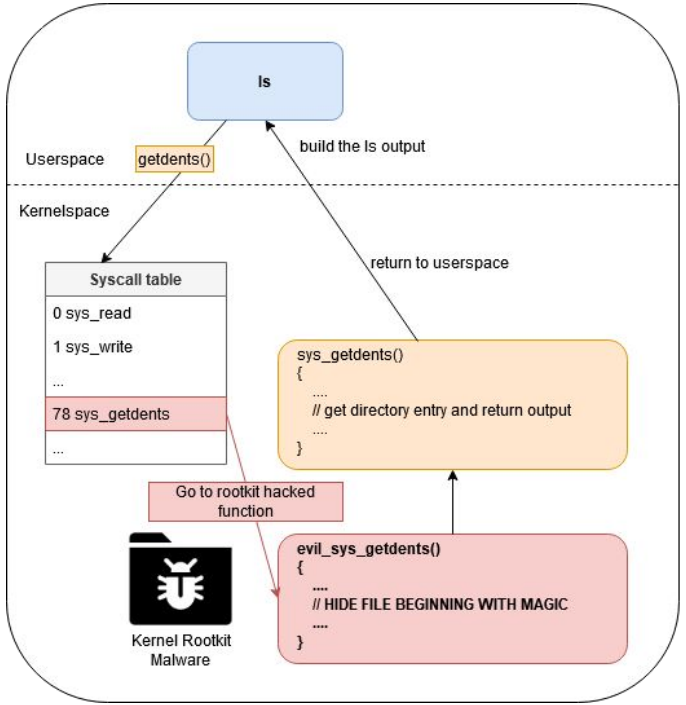
LKM rootkit deployment is constrained by privileges, security policy and kernel compatibility.

- Normal module loading requires effective **`CAP_SYS_MODULE`** in the initial user namespace (Requires Substantial Privilege)
- Loading may also be restricted by **signature enforcement**, **security policy**, or **`modules_disabled`**.
- The module must be compatible with the **target kernel's architecture,** configuration, and relevant ABI.

Method 1: Function-pointer Hooking

- **Hooking** is a technique for changing execution flow.
  - New hook registers address as location for a specific function
  - The replacement handler runs when the modified pointer is used!
- **Replace** a **syscall-table pointer** with the address of a **malicious handler.**
- User-space programs use the very same `getdents()` to find malicious files!

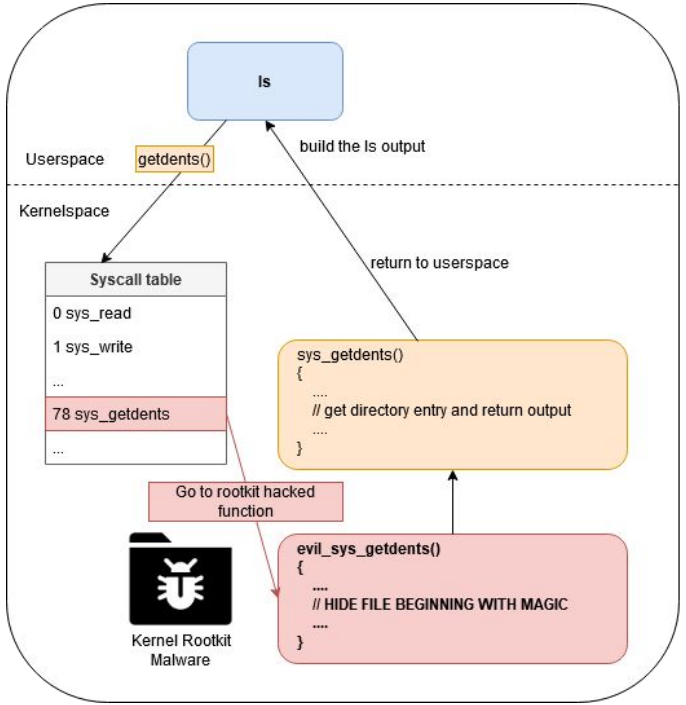
Traditional table-based dispatch model (Hooked)

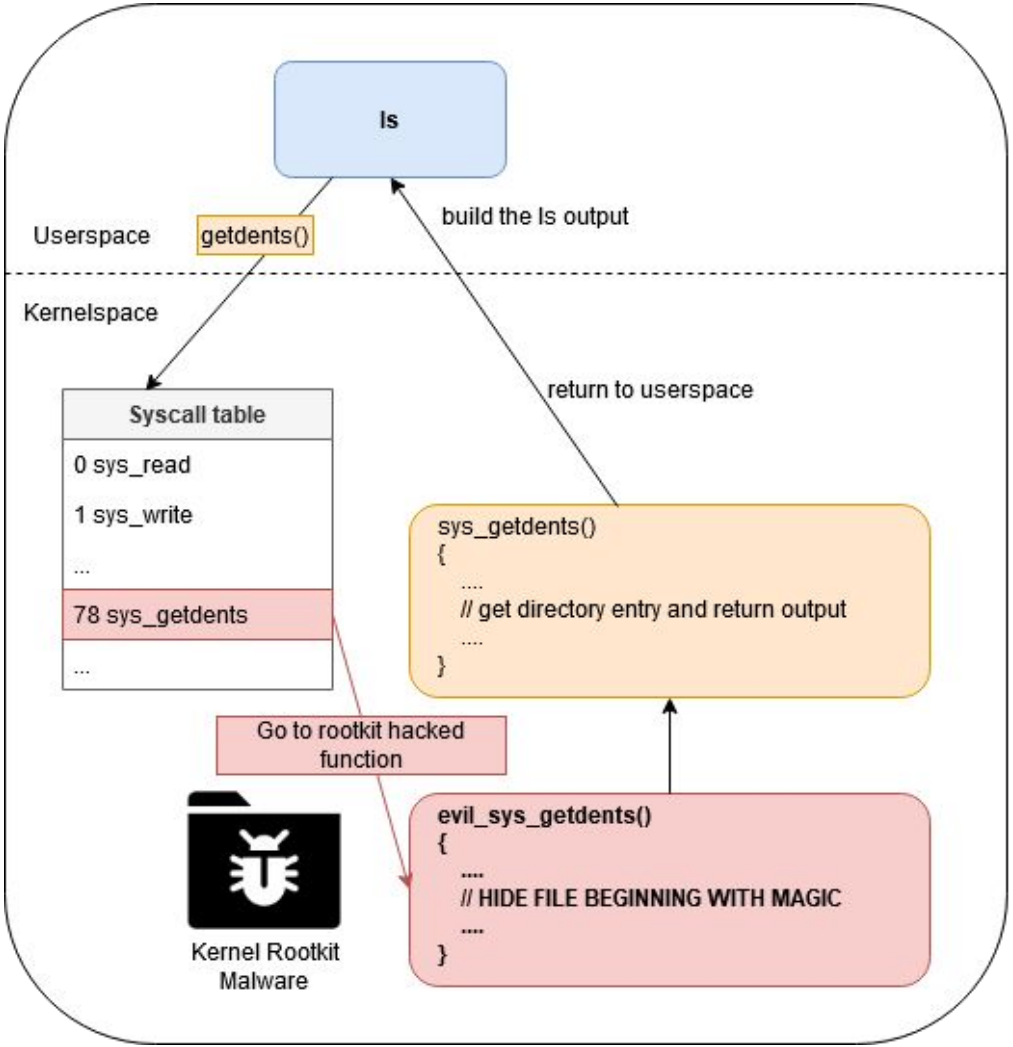


Function Pointer Hooking: Hooking the Syscall Table

1. Save existing entry in target call table (`sys_getdents()`)
2. Swap in new address for attacker hook (rootkit `evil_sys_getdents()`)
3. Intercept input & filter output
4. Execute the replacement handler &optionally calling the original function.

Traditional table-based dispatch model (Hooked)




ls
Userspace
getdents()
build the ls output
Kernelspace
return to userspace
Syscall table
0 sys_read
1 sys_write
...
78 sys_getdents
...
sys_getdents()
{
....
// get directory entry and return output
....
}
Go to rootkit hacked function
evil_sys_getdents()
{
....
// HIDE FILE BEGINNING WITH MAGIC
....
}
Kernel Rootkit Malware

Function-Pointer Hooking: Kernel Objects & Operation Tables

Redirecting execution through **kernel callbacks**

- Kernel Objects and Operation Tables contain **Function Pointers** that determine how operations are handled
  - `file_operations.iterate_shared` → Directory Enumeration
  - `seq_operations.show` → Generate records of interfaces (e.g. `/proc/net/tcp`)
- Rootkit replaces **callback pointer** with address of replacement handler
  - Handler can alter behaviour or filter results
  - Original Function *optionally* called, instructions remain unchanged
- This uses the **same pointer-redirection principle** as syscall-table hooking
- Replacing one of those callbacks can change what userspace sees!

Method 2: Run-time Kernel Code Patching

- **Run-Time Kernel Memory Patching** changes the Kernel's in-memory code/data while system is running!
- With **inline hooking**, the pointer can still point to the correct function address, but the code at that address has changed.
  - **Original function pointer** remains **unchanged**
  - Modified instructions can redirect code elsewhere! (commonly with `jmp`)
- However, Jumps, breakpoints, or changed bytes alone do not prove malicious hooking

In-Place Patch VS Inline Hooking

In-Place Patching	Inline Hooking
Patch implementation of original logic	Patch instructions that redirect execution
Modified behaviour lives in original code region	Modified behaviour lives in separate handler
Replacement logic must fit the available patch region	Redirection must fit the patch region

The Syscall Table can remain unchanged for Patching methods!

Run Time Kernel Mode Patching Diagram

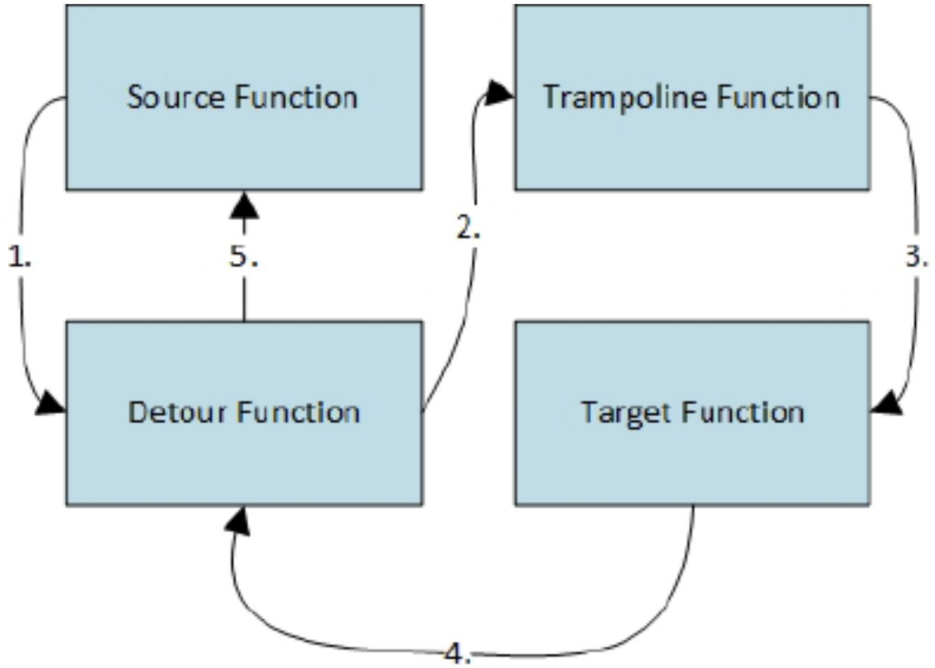


Fig. 3 Execution sequence of a Detours setup.

`ftrace`-based Hooking

`ftrace` is a built-in tracing framework for troubleshooting.

- Attackers use this to hook and intercept the execution of kernel functions
- Original: **Kernel Function Entry → Original Function**
- Modified**: Kernel Function Entry → `ftrace` callback → Replacement Handler**

Example: **`Singularity`** https://github.com/MatheuZSecurity/Singularity

kprobes-based Hooking

kprobes is a kernel-managed probing mechanism used for debugging and tracing

- Allows a handler to run when execution reaches an eligible kernel instruction
- Attackers can abuse probe handlers to inspect & alter control flow
- Original: **Kernel Function → Original Instructions → ...**
- Modified: **Kernel Function → Patched Breakpoint → Kprobe Handler → ...**

Example: **Reptile** https://github.com/f0rb1dd3n/Reptile

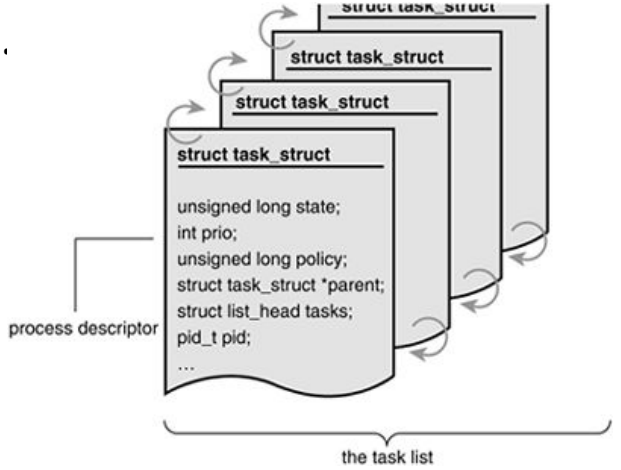
Method 3: Direct Kernel Object Manipulation (DKOM)

- **TLDR: Hiding without hooking; Modify Directly**
- Operating systems store internal bookkeeping information in kernel data structures.
  - Processes & threads/ Loaded kernel modules/ Open files & sockets/ Creds & permissions
  - Connected through lists/trees/tables
    - `task_struct` struct for storing process information
    - The kernel module list links `struct module` objects in a **doubly linked list**
  - Different kernel subsystems may contain different views of same underlying objects
- **Manipulate kernel data structure directly** without installing call hook
  - e.g. Unlink a module from the loaded-module list while leaving its code resident.

Process Hiding with DKOM

`EPROCESS` data structure (Windows) stores process information in the OS

- Linux equivalent: `task_struct`, unlinking `task_struct.tasks`, etc.
- Unlinking a task affects walkers of that structure, it does not necessarily hide it from every PID lookup or proc interface.
- Rootkit modifies pointer to hide information!
- Similar concept for Module hiding



Summary of Common LKM Rootkit Mechanisms

List of Common LKM Rootkit Mechanisms:

1. **Function Pointer Hooking** → Replace a callback to alter behaviour or filter reported objects.
2. **Run-Time Kernel Code Patching** → Changes the Kernel's in-memory code/data
3. **Direct Kernel Object Manipulation (DKOM)** → Remove module's entry from data structure being enumerated

Combining LKM Rootkit Techniques

A single rootkit might:

1. Use **function-pointer hooking** to intercept directory enumeration.
2. **Filter** the returned entries to hide selected files or processes.
3. Use **DKOM** to unlink its own module.
4. Use **inline hooking** for another reporting path.



Part IV: Rootkit Defence Tools & POCs



Linux LKM Rootkit Defence CLI Tools (Non-Exhaustive)

Detection Tool POCs & Demonstration:

1. **`lkmrkdet`** (POC #1 - Kernel Space + `client.py` program)
2. **`inodeyou-cpp`** (POC #2 - User Space)
3. **`unhide-tcp`**

Other detection, instrumentation, and hardening approaches:

1. **`chkrootkit`**
2. **`rkhunter`**
3. Linux Kernel Integrity Measurer (LKIM)
4. eBPF & **`bpftrace`** (Instrumentation)
5. **`/proc/sys/kernel/modules_disabled`** (Hardening)

Test Environment

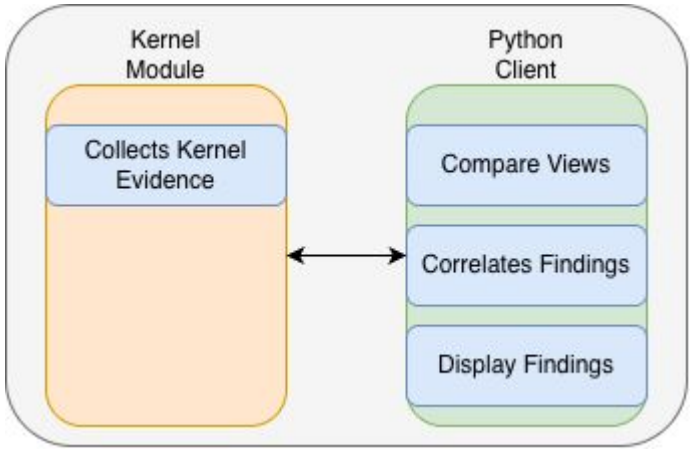
- **Ubuntu 24.04.4 LTS**
- `Linux ubuntu 6.8.0-146-generic #146-Ubuntu SMP PREEMPT_DYNAMIC Thu Sep  3 16:12:30 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux`



POC #1: `lkmrkdet` - Kernel Module + Userspace Client

`lkmrkdet` combines a **loadable kernel module (LKM)** with a **Python userspace client** to look for evidence of kernel rootkits.

Link: https://github.com/0xF41/lkmrkdet/tree/main



lkmrkdet

A Linux kernel rootkit detection tool that helps spot hidden processes and suspicious changes in the kernel. It uses a kernel module and a Python command-line client.

Tested on Ubuntu 24.04, x86-64, kernel `6.8.0-146-generic`.

Quick start

1. Download and build

Run these commands on the Linux machine you want to inspect:

```
sudo apt update
sudo apt install git build-essential python3 "linux-headers-$(uname -r)"
git clone https://github.com/0xF41/lkmrkdet.git
cd lkmrkdet
make module
```

The build creates `lkmrkdet.ko` for your running kernel. Rebuild it after a kernel update. Other kernel versions have not been validated.

2. Load the detector and scan

`lkmrkdet` Requirements

Requirements:

- `build-essential`
- `python3`
- `linux-headers-$(uname -r)`

1. Download and build

Run these commands on the Linux machine you want to inspect:

```
sudo apt update
sudo apt install git build-essential python3 "linux-headers-$(uname -r)"
git clone https://github.com/0xF41/lkmrkdet.git
cd lkmrkdet
make module
```

The build creates `lkmrkdet.ko` for your running kernel. Rebuild it after a kernel update. Other kernel versions have not been validated.

2. Load the detector and scan

```
sudo insmod ./lkmrkdet.ko
sudo python3 client.py scan
```

The default scan checks kernel modules, hidden processes, filesystem and network reporting, system-call handlers, and tracing hooks.

3. Unload when finished

```
sudo rmmod lkmrkdet
```

`lkmrkdet` Usage & Features

Features:

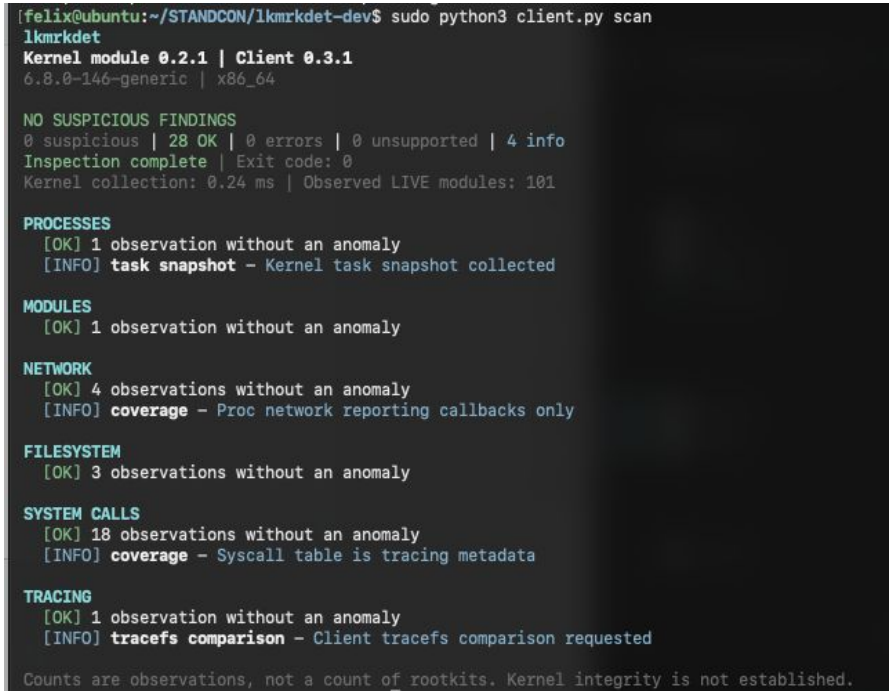
I. Detect **hidden-process** candidates
II. Detect **hidden-module** registration discrepancies
III. Detect **hooks affecting network reporting**
IV. Detect **filesystem-enumeration hooks**
V. Detect **system-call tampering**
VI. Correlate suspicious **tracing** evidence



`lkmrkdet` Demo: Clean System

Run test after inserting **`lkmrkdet.ko`** for the first time

- **`[OK]`** and **`[INFO]`** messages
- **No anomalies detected** across Processes, Modules, Network, Filesystem, Syscalls, Tracing



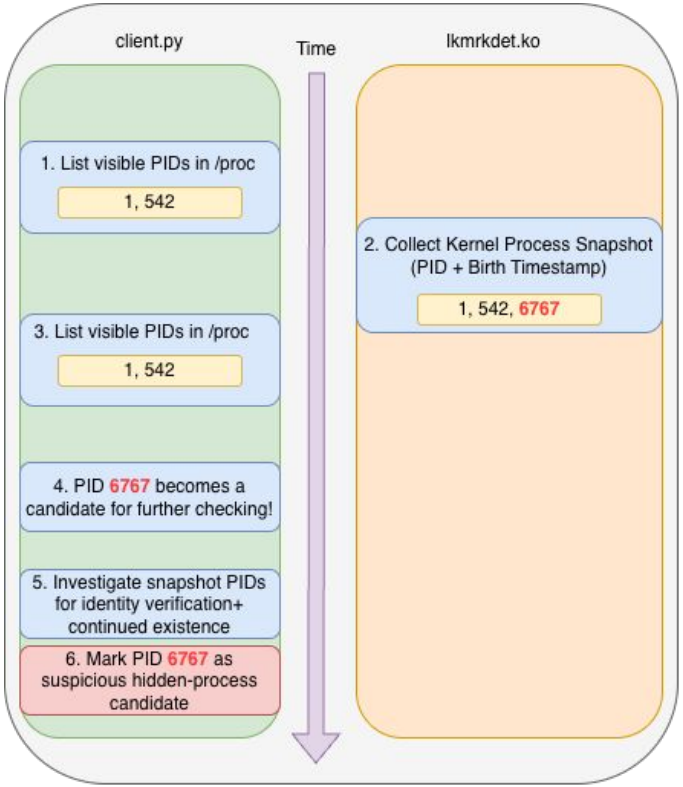
I: Detect hidden-process candidates with `lkmrkdet`



I: Detect hidden-process candidates with `lkmrkdet`

A process may disappear from **`/proc`** while remaining in the **kernel's task list**

- Kernel module **`lkmrkdet.ko`** collects process leaders and birth timestamps
- Python client **`client.py`** enumerates **`/proc/<pid>`** entries before and after collection
- Compare within the same PID namespace

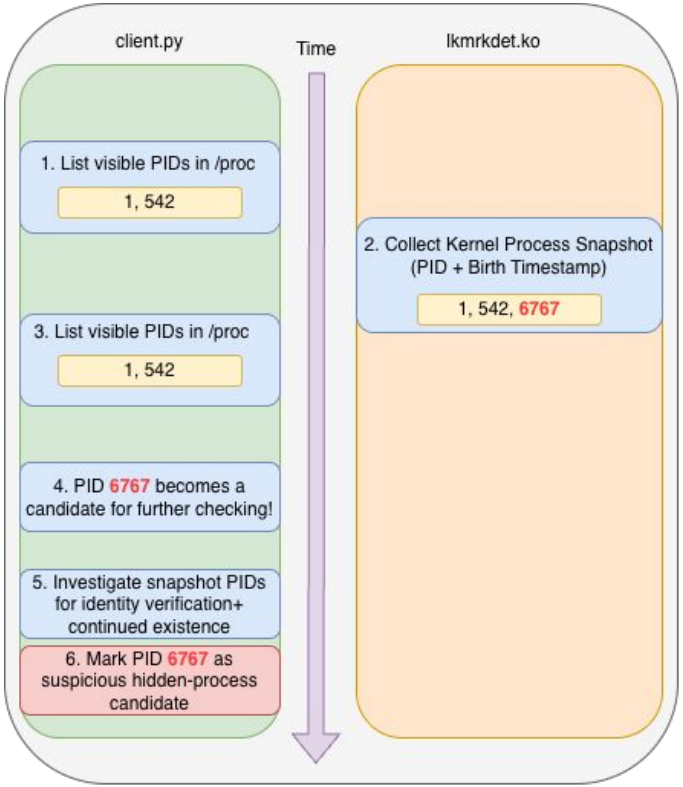


I: Detect hidden-process candidates with `lkmrkdet`

Verify identity before reporting a hidden-process candidate

- Try **`/proc/6767/stat`**.
- If **readable**, compare process start time with the start time in kernel snapshot
- If **not readable** (Lookup returns **`ENOENT`**/**`ESRCH`**) → Takes another kernel snapshot and requires the same PID + exact birth identity

Limitation: Task-list removal can evade this comparison!
(Both snapshots come from the same kernel)



I: Detect hidden-process candidates with `lkmrkdet`

Caraxes rootkit observation

- Filter out `/proc` directory listing
- Still found process in **kernel task snapshot**!
- Direct lookup confirms the same process start time → PID is not reused

```
SUSPICIOUS EVIDENCE FOUND
1 suspicious | 0 OK | 0 errors | 0 unsupported | 1 info
Inspection complete | Exit code: 1
Kernel collection: 0.11 ms | Observed LIVE modules: 103

PROCESSES
  [INFO] task snapshot - Kernel task snapshot collected
  [SUSPICIOUS] PID 4121 - Missing from proc enumeration
    Direct proc lookup matches the process start time; both enumerations omit this PID.
```

II. Detect hidden-module registration discrepancies with `lkmrkdet`



II. Detect hidden-module registration discrepancies with `lkmrkdet`

`lkmrkdet` compares two kernel registration structures that normally agree and flags LIVE modules present in only one view.

1) **Live Kernel Module-List Entries** → User-visible presentations
   - Reflected by **`lsmod`**, **`/proc/modules`**
   - Exclude built-in and transitional objects
   - Record executable text ranges for callback ownership checks
2) **Typed Module `kobjects`**
   - Reflected in kernel's object system under **`/sys/module`**

Note: Comparison of **kernel registration structures** beneath the **`lsmod`**, **`/proc/modules`**, **`/sys/module`** views!

II. Detect hidden-module registration discrepancies with `lkmrkdet`

Kernel module list	Module kobjects	Interpretation
Present	Present	Registrations agree
Present	Missing	Possible sysfs/kobject hiding
Missing	Present	Possible module-list hiding
Missing	Missing	This comparison cannot see the module

A rootkit that removes a module from both registration structures can evade this check. E.g. **Singularity** Rootkit evades this!

II. Detect hidden-module registration discrepancies with `lkmrkdet`

Diamorphine Rootkit Results:

- **Found** through **`kobject`** view;
- **Missing** from **live module-list view**!
- Detected as **`live_kobject_missing_module`**

```
MODULES
  [SUSPICIOUS] diamorphine – Module kobject missing from the module list
    A LIVE module is present in only one registry. Recheck lifecycle and corroborate.
    Address: 0xffffffffc092b000 | Owner: diamorphine (unlisted)
```

III. Detect hooks affecting network reporting with `lkmrkdet`



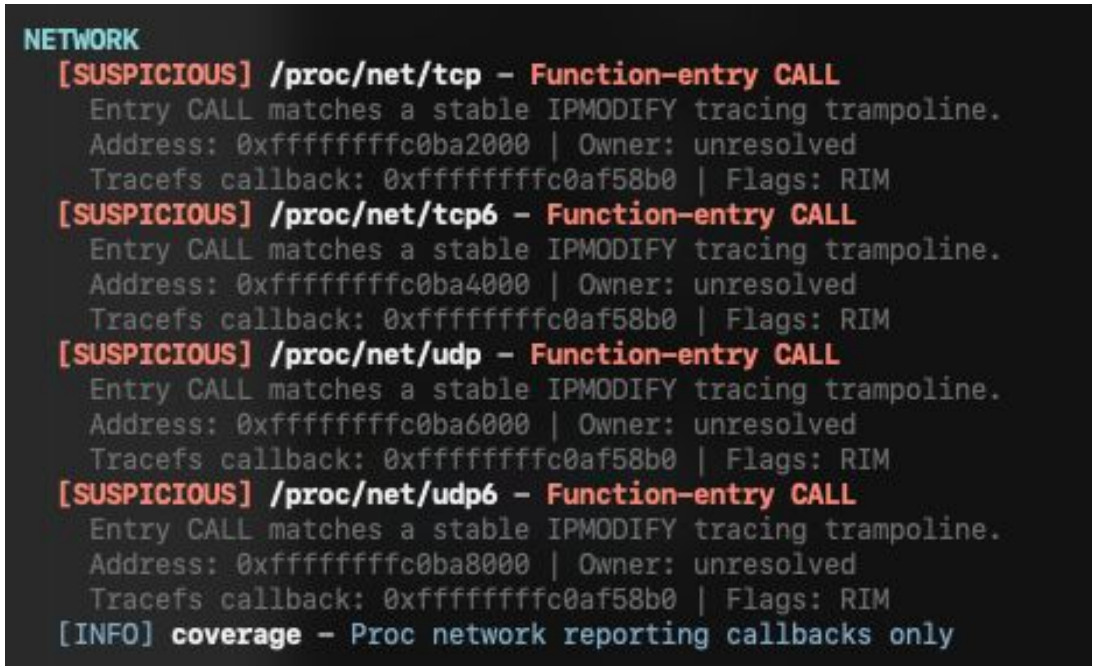
III. Detect hooks affecting network reporting with `lkmrkdet`

- **`netstat`** relies on information presented through **`/proc/net`**
  - Rootkit may hide connection by changing kernel's reporting path (socket still exists!)
- **`ss`** primarily uses socket-diagnostic **`netlink`**
- Open the **`/proc`** network-reporting endpoints
  - **`/proc/net/tcp`**, **`tcp6`**, **`udp`**, **`udp6`**
  - Each file uses kernel callback called **`seq_operations.show`** to format entries
  - The detector **reads the callback attached to each opened endpoint**, then **compare** with expected **TCP/UDP reporting function**
- Also **examine selected bytes at function entry** for suspicious branch patterns (for inline hooking)

III. Detect hooks affecting network reporting with `lkmrkdet`

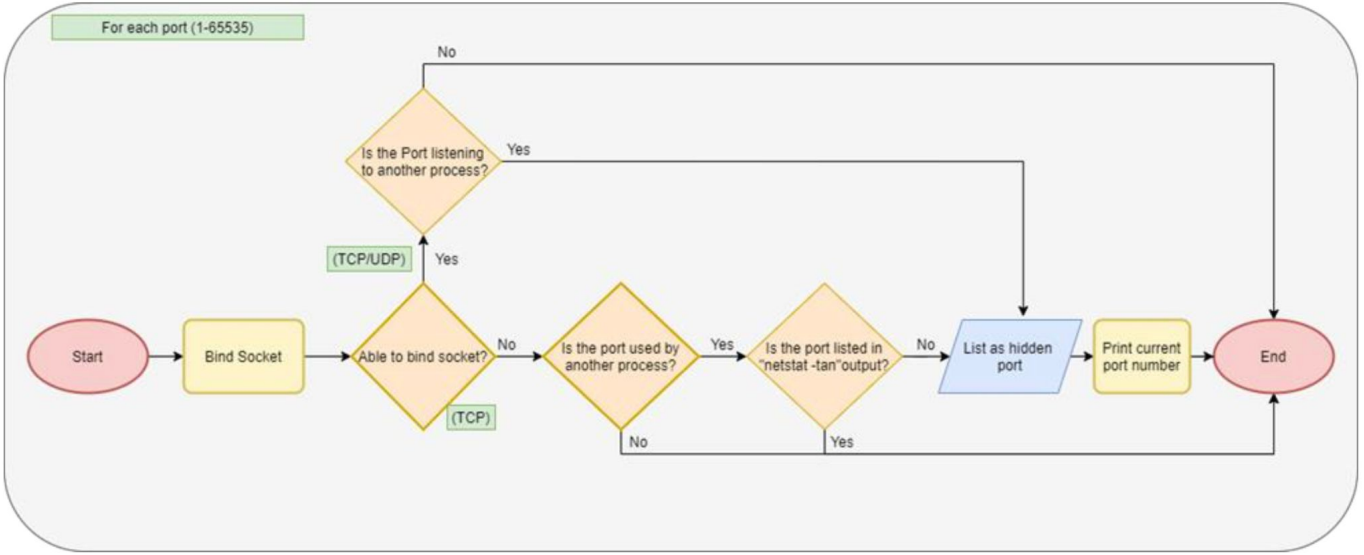
Singularity Rootkit Observations:

- The **connection remained active & usable** while its listing disappeared.
- `lkmrkdet` detected suspicious changes to the kernel's network-reporting path.
- Can **cross-check** for hidden-port discrepancies using `unhide-tcp`.



unhide-tcp

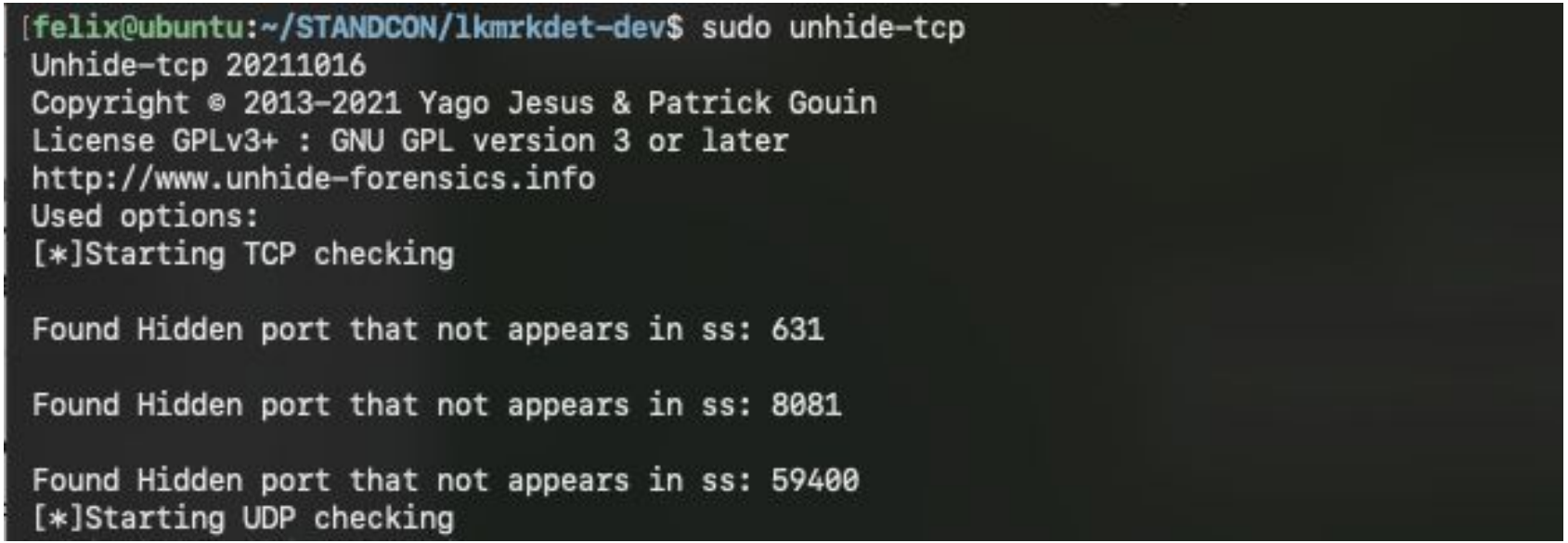
unhide-tcp attempts local socket operations across ports 1–65535 and compares the results with reported sockets.



TLDR: A socket hidden from a listing may still reveal itself through network-stack behaviour.

unhide-tcp

unhide-tcp reported multiple port discrepancies, including controlled TCP/8081 listener



IV: Detect filesystem-enumeration hooks with `lkmrkdet`

IV: Detect filesystem-enumeration hooks with `lkmrkdet`

Inspects **Directory Enumeration Paths**

- Inspect directory iteration callbacks for `/`, `/proc` and `/sys`
- Compare selected callbacks with expected handlers and executable owners
  - Check if **callbacks point to expected kernel functions**/observed executable region (Hooks?)
- Inspect selected `getdents64` syscall paths
- Correlate available `ftrace` redirection evidence
  - If an entry `CALL` targets a tracing trampoline, correlate it with existing `ftrace` registrations.
  - Explains redirection but does NOT prove malicious intent itself

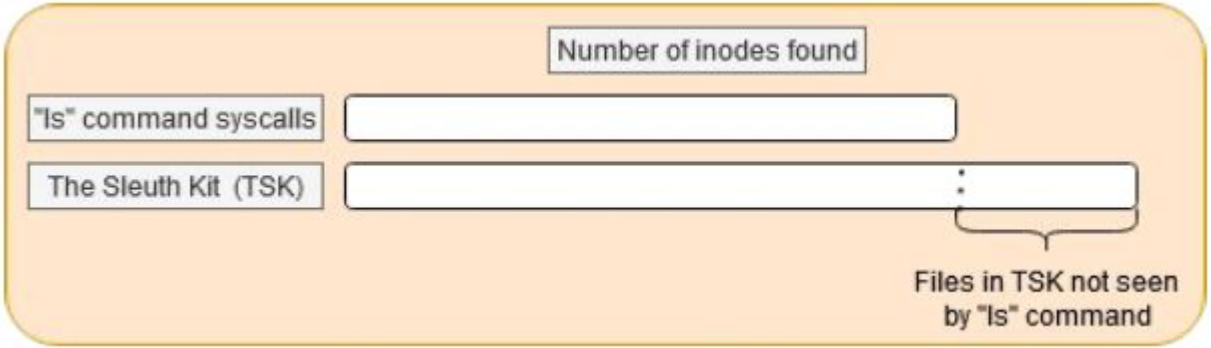
POC #2: `inodeyou-cpp`

- GitHub Repo: https://github.com/0xF41/inodeyou-cpp
- **`inodeyou-cpp`** is a forensic tool to **detect files and directories that may be hidden by malware** on Unix-like systems.
  - Reports **inode discrepancies** between a **TSK-parsed filesystem view** and the **mounted-filesystem view**.
- Uses **The Sleuth Kit (TSK)** to parse supported filesystem formats.

`inodeyou-cpp` Mechanism

`inodeyou-cpp` compares inode-and-path inventories

1) Finding inodes visible to the disk's view of the file system (with `read()`) via TSK library - **TSK Parsed Filesystem View**
2) Finding inodes visible from the user's view of the **mounted file system** (with **`getdents64()`**, **`stat()`**, etc.).



`inodeyou-cpp` demo

Tested with **Caraxes** Rootkit:

- Caraxes caused directory entries to disappear from the mounted view (Entries containing **`caraxes`**)
  - An omitted parent directory also prevents the mounted walk from discovering its descendants!
- TSK still recovered their corresponding paths!

`inodeyou-cpp` demo

Tested with **Caraxes** Rootkit:

```
felix@ubuntu:~/STANDCON/inodeyou-cpp$ sudo ./inodeyou /dev/sda2 ~
 [*] TSK inventory: 1362 inodes.
 [*] Mounted inventory: 1362 inodes.
 [*] No differences found between TSK and filesystem inode/path inventories.
felix@ubuntu:~/STANDCON/inodeyou-cpp$ sudo insmod /home/felix/STANDCON/rootkits/caraxes/caraxes.ko
felix@ubuntu:~/STANDCON/inodeyou-cpp$ sudo ./inodeyou /dev/sda2 ~
 [*] TSK inventory: 1362 inodes.
 [*] Mounted inventory: 1287 inodes.
 [!] RAW-ONLY: hidden-entry candidate | inode=1180102
     Path: "/home/felix/STANDCON/rootkits/caraxes"
 [!] RAW-ONLY: hidden-entry candidate | inode=1180103
     Path: "/home/felix/STANDCON/rootkits/caraxes/.git"
 [!] RAW-ONLY: hidden-entry candidate | inode=1180104
     Path: "/home/felix/STANDCON/rootkits/caraxes/.git/branches"
 [!] RAW-ONLY: hidden-entry candidate | inode=1195273
     Path: "/home/felix/STANDCON/rootkits/caraxes/.git/index"
 [!] RAW-ONLY: hidden-entry candidate | inode=1195412
     Path: "/home/felix/STANDCON/rootkits/caraxes/.git/hooks"
```

IV: Detect filesystem-enumeration hooks with `lkmrkdet`

Limitation: This does not enumerate every directory or discover unknown hidden pathnames!

- These are **hook and callback checks**, not the disk-versus-mounted-filesystem comparison used by **`inodeyou-cpp`**!
- Use **`inodeyou-cpp`** to investigate object-level filesystem discrepancies.
- Direct directory-callback replacement was not validated with a rootkit on our tested kernel (6.x)
  - Potentially detect older rootkits (Source-reviewed Candidates): Suterusu, Adore-ng v2.5 — kkamagui fork, kprochide

V: Detecting System-Call Tampering with `lkmrkdet`

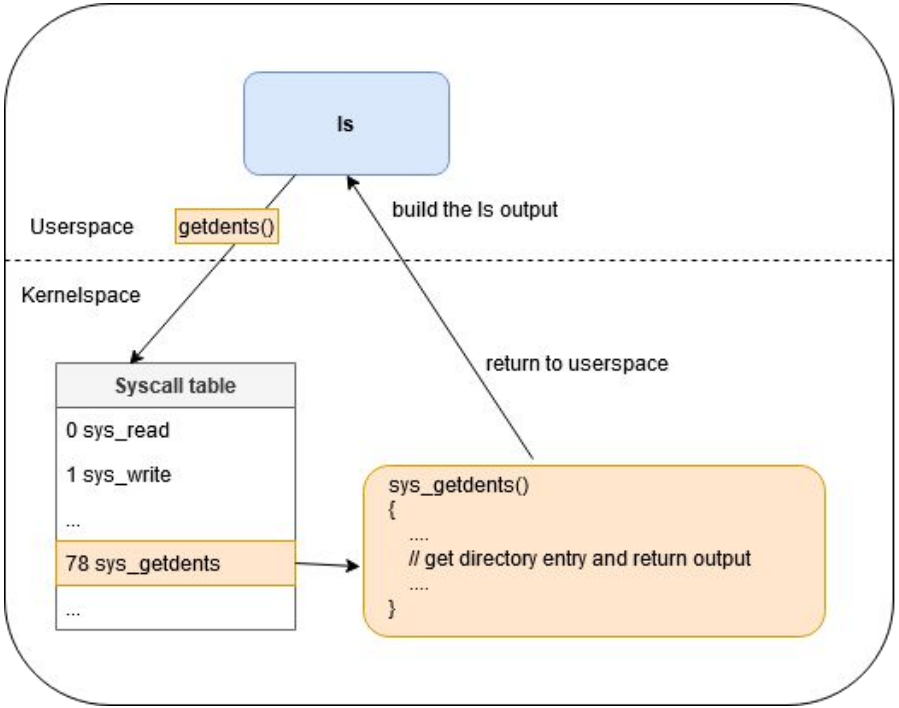


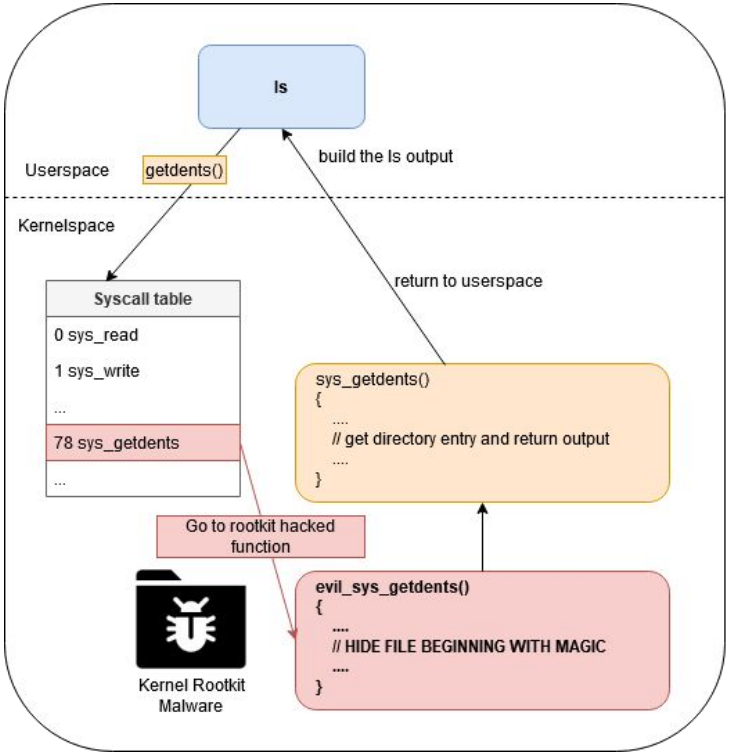
V: Detecting System-Call Tampering with `lkmrkdet`

`lkmrkdet` checks for system-call tampering in two ways:

1. **Table-pointer Inspection**
   - Read selected native syscall-table entries in Kernel's syscall table
   - **Compare** each observed pointer with its independently resolved named syscall handler
2. **Handler-entry Inspection**
   - The table may not be the path Linux actually uses to dispatch system calls. So the detector also l**ooks at the first bytes** of selected handler functions. (Pattern-based)
     - **Examine bounded bytes** at selected handler entries
     - **Recognise supported branch** and breakpoint patterns
     - **Evaluate branch destinations** against observed executable ownership

Recap of Traditional Filesystem-Hiding Mechanisms





V: Detecting System-Call Tampering with `lkmrkdet`

Diamorphine rootkit observation

- Changed **`getdents`**, **`getdents64`**, **`kill`** **table pointers**
- **`lkmrkdet`** flagged all three!
- Matching headers identify the table as tracing metadata (not live dispatch)

```
SYSTEM CALLS
  [INFO] coverage - Syscall table is tracing metadata
  [SUSPICIOUS] table:getdents64[217] - Unexpected handler pointer
    Observed pointer differs from the named expected handler. Authorised redirection must be
    considered.
    Address: 0xffffffffc092b010 | Owner: diamorphine (unlisted)
  [SUSPICIOUS] table:kill[62] - Unexpected handler pointer
    Observed pointer differs from the named expected handler. Authorised redirection must be
    considered.
    Address: 0xffffffffc092b710 | Owner: diamorphine (unlisted)
  [SUSPICIOUS] table:getdents[78] - Unexpected handler pointer
    Observed pointer differs from the named expected handler. Authorised redirection must be
    considered.
    Address: 0xffffffffc092b390 | Owner: diamorphine (unlisted)
```

Note: Table tampering detected; live syscall interception was not demonstrated.

VI: Correlate suspicious tracing evidence with `lkmrkdet`



VI: Correlate suspicious tracing evidence with `lkmrkdet`

`lkmrkdet` uses existing **`ftrace`** information to identify suspicious code changes

1. Read a snapshot of **`enabled_functions`**, which reports tracing registrations.
2. Inspect selected kernel pointers and function-entry instructions.
3. Read **`enabled_functions`** again to check whether the selected registrations remained **stable**.
4. Match entry-**`CALL`** destinations with reported trampoline addresses to obtain supporting callback evidence.

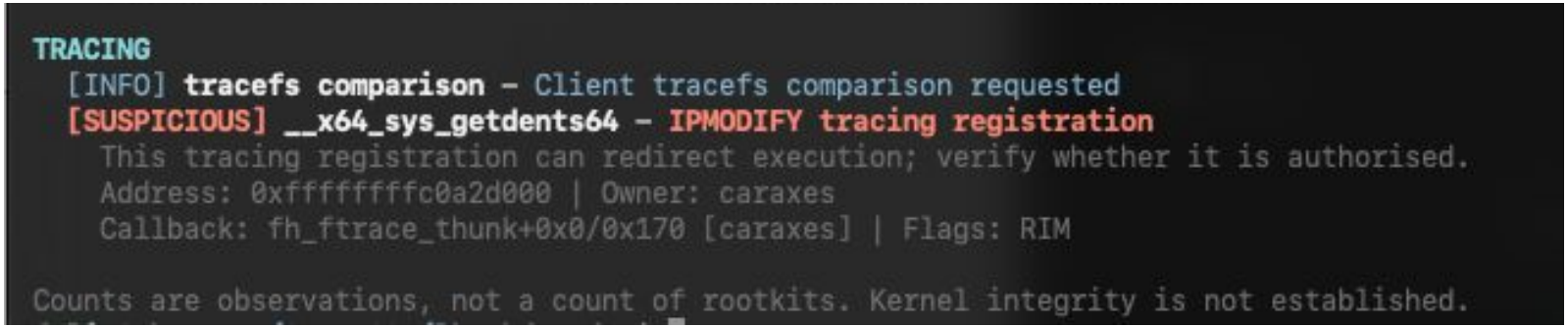
`IPMODIFY` registrations can redirect function execution. Legitimate software can also use them.

We correlate an entry **CALL** with a reported callback only when the selected registration is stable and the trampoline address matches exactly. Otherwise, we retain **uncertainty**.

VI: Correlate suspicious tracing evidence with `lkmrkdet`

Caraxes & Singularity Rootkit observation

- **Caraxes** callback includes **`[caraxes]`**, providing named attribution
- **Singularity** callback is numeric; preserves address without guessing owner
- Legitimate tracing/livepatch can also produce similar evidence



Summary of `lkmrkdet` mechanisms

I. **Processes**: task snapshot ↔ proc enumeration
II. **Modules**: module list ↔ module kobjects
III. **Network**: observed callback/entry ↔ expected handler and tracing evidence
IV. **Filesystem**: selected directory callback ↔ expected handler
V. **Syscalls**: table pointer/handler entry ↔ expected symbol and recognised patterns
VI. **Tracing**: entry CALL ↔ stable tracing registration

Summary of Tested LKM Rootkits

Sample	Controlled behaviour observed	`lkmrkdet` evidence	Important limitation
Caraxes	Process and filesystem enumeration hiding	Hidden-PID candidate; enumeration-handler and tracing findings	Module hiding disabled in tested configuration
Diamorphine	Module-list concealment; syscall-table changes	Module registration discrepancy; three table-pointer mismatches	File/process hiding not observed in tested environment
Singularity	Process, filesystem, and TCP IPv4 reporting hiding	Hidden-PID candidate; network and enumeration-hook evidence	Module registration comparison missed the concealed module

Part V: Where Trust Ends



Shared-Kernel Limitations

Scenario: A detector loads in an **already-compromised environment.**

- **Baseline Contamination**
  - If a detector uses baseline comparisons, existing bad modifications may be baselined by the security tool!
- **Manipulated Observations**
  - Kernel APIs and Object Lists used by the detector may already be hooked/altered
- **Detector Interference**
  - A rootkit with sufficient kernel control may interfere with the detector's loading/execution/memory/results
- **Limited Assurance**
  - The detector may miss inconsistencies, resulting in **false negatives**.



Key Takeaways

Hidden from enumeration does not mean absent or inactive!

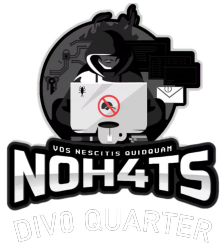
- Seen in **`unhide-tcp`** and **`inodeyou-cpp`**

When the kernel lies, inconsistencies may still expose it.!

- Agreement between observations made through that same kernel is not proof that it is trustworthy
- A detector may find **evidence** of compromise, but a negative result cannot establish kernel integrity!







Attendance



‹/talk›

Thank you for your kind attention! **<questions>**

LinkedIn



References

- https://www.scaler.com/topics/linux-kernel-architecture/
- https://internals-for-interns.com/posts/linux-kernel-syscalls/
- https://www.wiz.io/blog/linux-rootkits-explained-part-2-loadable-kernel-module
- https://www.researchgate.net/publication/319868952_The_Evolution_of_Process_Hiding_Techniques_in_Malware_-_Current_Threats_and_Possible_Countermeasures#pf5
- https://attack.mitre.org/techniques/T1547/006/
- https://attack.mitre.org/detectionstrategies/DET0377/